lepidoptera: A Domain-Agnostic Engine for Causal Simulation with Mixed Variable Types, Vine Copula Dependence, and Individual-Level Counterfactual Propagation

Danilo A. Sarti

Lecturer in AI and Machine Learning

Royal College of Surgeons in Ireland, Dublin, Ireland

danilosarti@rcsi.ie

https://github.com/daniloasarti/lepidoptera

Preprint — October 16, 2026

Abstract

lepidoptera is an open-source R and Python package for simulation from structural causal models with heterogeneous variable types. The package is built around a *typed mechanism* architecture, which separates the causal graph from the generating rule at each node and exposes each rule as an inspectable, serialisable object. A pre-built mechanism library covers all nine combinations of continuous, categorical, and ordinal parent and child types, removing the need for users to implement type-specific sampling from scratch. A vine copula mechanism estimates the full joint dependence structure from observed data using pair-copula constructions (Bedford and Cooke, 2002; Aas et al., 2009) and the Rosenblatt conditional-simulation algorithm (Rosenblatt, 1952), supporting over twenty copula families through **rvinecopulib** (Nagler and Vatter, 2022). Individual-level counterfactual queries are supported through *partial propagation*, which freezes the random draws of unaffected nodes and replays only the downstream consequences of an intervention, thereby separating the individual treatment effect (ITE) estimand from the population-level average treatment effect (ATE) without additional model assumptions. An extension layer allows any pre-fitted predictive model to serve as a generative mechanism via a unified callable interface. Three worked illustrations spanning clinical trial planning, emergency-ward capacity analysis, and epidemiological mediation analysis demonstrate the package's scope.

1 Introduction

Simulation from structural causal models (SCMs) plays a central role across applied statistics. Monte Carlo power calculations for clinical trials routinely demand realistic synthetic patient populations that honour complex dependencies among continuous biomarkers, ordinal performance scores, and categorical treatment assignments (Morris et al., 2019; Burton et al., 2006). Digital twinning for hospital operations and personalised medicine requires

the ability to intervene on individual pathways and measure downstream consequences at the level of individual units rather than population averages (Laubenbacher et al., 2024). Epidemiological mediation analyses call for joint interventions on both exposures and mediators, where the individual-level quantities needed to decompose effects into direct and indirect components (VanderWeele, 2015; Pearl, 2001) are conceptually distinct from the population-level estimands produced by simple rerandomisation (Robins and Greenland, 1992). Despite this diversity of applications, the computational problem underlying all three scenarios is the same: simulate from a directed acyclic graph (DAG) (Pearl, 2009; Spirtes et al., 2000) whose nodes span heterogeneous measurement scales, apply one or more interventions, and distinguish between what happens on average across a population and what would have happened to a specific individual (Rubin, 1974; Neyman, 1990).

Existing software addresses parts of this problem, but no single package handles all of it. **simcausal** (Sofrygin et al., 2017) and **simDAG** (Denz and Timmesfeld, 2026) offer the most comprehensive DAG-based simulation facilities currently available in R, and both packages have been widely adopted. Their designs, however, treat a mechanism primarily as a named node type or a formula: a string or expression whose generating logic is baked into the package internals and cannot be replaced in isolation. This precludes the clean intervention semantics that Pearl's modularity assumption (Pearl, 2009) demands and makes it cumbersome to substitute a data-fitted copula or a random-forest mechanism in place of a parametric equation without re-specifying the entire node. Neither package natively supports vine copula dependence modelling or individual-level (counterfactual) propagation. **DagSim** (Al Hajj et al., 2023) addresses the restriction on variable types and functional forms in its Python implementation by routing all generating logic through user-supplied functions, but provides no pre-built mechanism library, no vine copula backend, and no partial propagation. The R package **R6causal** (Tikka, 2024) supports counterfactual simulation for parametric SCMs with discrete variables but does not cover the mixed-type workflow described above.

lepidoptera is designed to fill this gap. The package name reflects the modular, compositional logic of biological taxonomy: just as a lepidopterist builds an identification key from composable, observable features, **lepidoptera** builds a data-generating process from composable, observable mechanisms—each typed, replaceable, and independently auditable. The central design decision is the strict separation of *graph* (which variables exist and who depends on whom) from *mechanism* (how each variable is generated given its parents), a separation grounded in Pearl's modularity assumption for SCMs (Pearl, 2009; Peters et al., 2017). This separation has four direct consequences. First, a pre-built mechanism library covers all nine parent-type/child-type combinations arising from the Cartesian product of {continuous, categorical, ordinal}; users specify parameters and the package handles type coercion, domain validation, and the correct sampling distribution. Second, a vine copula mechanism learns the full joint dependence structure from observed data and registers a conditional-simulation rule at the specified node, so that data-driven dependence can be incorporated without replacing the rest of the system. Third, `propagate()` with `mode = "partial"` freezes all unaffected random draws and recomputes only the downstream subgraph, enabling ITE estimation from the same synthetic population as the ATE. Fourth, any R or Python callable can serve as a mechanism through three wrapper functions that add type validation and serialisation support, making it straightforward to incorporate random forests, gradient-boosted trees, or language-model outputs into a causal simulation without any adapter code. Systems serialise to JSON for cross-language portability, so a model defined in R can be re-simulated in Python and vice

versa.

The remainder of this paper is organised as follows. Section 2 surveys related software and positions **lepidoptera** within the existing landscape. Section 3 introduces the formal framework. The mechanism library and its R interface are described in Section 4, followed by the simulation and propagation algorithms in Section 5. The vine copula mechanism is treated in depth in Section 6, and the predictive model integration layer is described in Section 7. Three illustrations are given in Section 8. Computational details and package availability are covered in Section 9, and the paper closes with a discussion in Section 10.

2 Related software

The landscape of DAG-based simulation in R is anchored by **simcausal** (Sofrygin et al., 2017) and, more recently, **simDAG** (Denz and Timmesfeld, 2026). **simcausal** provides a domain-specific language for specifying non-parametric SEMs for longitudinal data with time-varying treatments, censoring, and monitoring processes, motivated by the marginal structural model framework of Robins et al. (2000). The package computes treatment-specific means and related causal quantities for evaluating targeted learning estimators (van der Laan and Rose, 2018) and has been widely used in simulation benchmarks for that literature. **simstudy** (Goldfeld and Wujciak-Jens, 2020) is another widely-used R package for study-design simulation, supporting a broad range of outcome distributions but without an explicit causal graph layer. Its design centres on a temporal node index and a shared R environment evaluator, which is convenient for longitudinal settings but makes cross-sectional DAGs with heterogeneous mechanisms cumbersome to specify: there is no typed mechanism object, no vine copula, and no native support for individual-level counterfactual queries. **simDAG** (Denz and Timmesfeld, 2026) addresses several of these limitations. Published in the Journal of Statistical Software in 2026, it offers a clear and expressive node-type vocabulary—`node_gaussian`, `node_binomial`, `node_multinomial`, `node_cox`, and others—that directly supports generating binary, categorical, count, and time-to-event data within a DAG. Like **simcausal**, however, **simDAG** represents mechanisms as named strings rather than first-class objects, which means that an intervention at any node must be expressed through the node-type system rather than by mechanism replacement. Vine copula dependence modelling and individual-level counterfactual propagation are absent.

For Bayesian network structure learning, **bnlearn** (Scutari, 2010) is the reference implementation in R, providing both constraint-based and score-based structure learning algorithms (Chickering, 2002; Spirtes et al., 2000). Simulation from a **bnlearn** network is restricted to linear Gaussian or discrete conditional probability table mechanisms; ordinal variables, vine copulas, and partial propagation are not supported, reflecting the package's primary orientation towards inference rather than simulation. **lavaan** (Rosseel, 2012) and **OpenMx** (Neale et al., 2016) implement structural equation models and support simulation, but both are restricted to linear Gaussian mechanisms and do not accommodate the mixed-type settings common in applied biostatistics (Lauritzen and Wermuth, 1989). **dagitty** (Textor et al., 2016) provides DAG visualisation and d-separation analysis (Verma and Pearl, 1988) without generating data, and serves as a natural companion to **lepidoptera** for structural identification prior to simulation.

In Python, **DagSim** (Al Hajj et al., 2023) is the closest counterpart, motivated by the observation that existing tools remain confined to settings with relatively simple variable types and functional forms. **DagSim** removes this restriction by routing all generating logic through user-supplied functions specified in YAML. **lepidoptera**'s Python implementation

follows a similar philosophy but adds typed mechanism validation, a pre-built library, vine copula integration, and partial propagation. **CausalNex** (Beaumont et al., 2021) and **DoWhy** (Sharma and Kiciman, 2020) focus on structure learning and causal effect estimation rather than generative simulation.

On the methodological side, Karvanen et al. (2024) present an algorithm for exact counterfactual simulation in parametric SCMs, handling background variables that affect multiple observed variables. Their method is more general than **lepidoptera**'s partial propagation in that it accommodates non-Gaussian error structures explicitly, but it requires fully parametric models and does not target the mixed-type simulation workflow. For counterfactual query *identification*, the **cfid** package (Tikka et al., 2021) implements the ID* and IDC* algorithms; it does not simulate data and is complementary to **lepidoptera**.

Table 1 summarises the feature comparison across the tools discussed above.

Table 1: Feature comparison of causal simulation tools (October 2026). Mixed types: native support for continuous, categorical, and ordinal in any parent-child combination. Copula: vine copula mechanism. CF prop: individual-level partial propagation. Typed mech: mechanism as a first-class object. R/Py: availability in both languages.

Package	Lang.	Mixed types	Copula	CF prop.	Typed mech.
simcausal (Sofrygin et al., 2017)	R	Partial	No	No	No
simDAG (Denz and Timmesfeld, 2026)	R	Yes	No	No	No
bnlearn (Scutari, 2010)	R	Partial	No	No	No
R6causal (Tikka, 2024)	R	Partial	No	Partial	No
lavaan (Rosseel, 2012)	R	No	No	No	No
DagSim (Al Hajj et al., 2023)	Python	Yes	No	No	No
lepidoptera (this paper)	R+Py	Yes	Yes	Yes	Yes

3 The lepidoptera framework

The formal framework rests on three concepts: variable types, mechanisms, and systems.

Every feature V in a **lepidoptera** system has a *type* $\tau(V) \in \{\texttt{continuous}, \texttt{categorical}, \texttt{ordinal}\}$ and a domain $\mathcal{D}(V)$. Continuous features have a real-valued domain, possibly bounded; categorical features take values in a finite unordered set of levels; ordinal features take values in a finite totally ordered set. Type information is declared at system-construction time and checked at `finalize_system()`, so type errors surface immediately rather than at simulation time.

A *mechanism* for feature V_i with parent set $\mathbf{Pa}_i \subseteq \mathcal{V} \setminus \{V_i\}$ is a triple $(\text{fn}_i, \boldsymbol{\tau}_{\mathbf{Pa}}, \tau_i)$, where fn_i is a measurable function mapping parent realisations and exogenous noise to a value in $\mathcal{D}(V_i)$, $\boldsymbol{\tau}_{\mathbf{Pa}}$ is the declared vector of parent types, and τ_i is the declared output type. In R, a mechanism is a named list carrying these fields alongside a serialisable parameter record; in Python it is a dataclass with the same fields. Because the mechanism is a first-class object, any single mechanism can be extracted, inspected, replaced, or overridden in isolation—the rest of the system is unaffected.

A *system* $\mathcal{S} = (\mathcal{V}, \mathcal{E}, \{\mathcal{M}_i\})$ consists of a finite set of typed features, a set of directed edges $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ whose induced graph G is acyclic, and a mechanism for each feature whose declared parent types are consistent with the actual types of the corresponding parents. The function `finalize_system()` verifies five invariants before any simulation can proceed: that G is acyclic, that every feature has exactly one mechanism, that all

referenced names exist in $\mathcal{V}$, that declared parent types match actual types, and that no orphaned mechanisms exist. Systems serialise to JSON via `export_system()` and can be re-imported in either language via `import_system()`.

An *intervention* specifies a target set $\mathcal{T} \subseteq \mathcal{V}$, a kind drawn from {`do_fix`, `do_distribution`, `shift`, `mechanism_override`}, and kind-specific parameters. `do_fix`$(V_i = v_0)$ replaces the mechanism at V_i with the constant v_0 and severs all incoming edges, implementing Pearl's do-operator (Pearl, 1995). `shift`(V_i, δ) composes the existing mechanism with a measurable transformation δ of its output domain. `do_distribution` and `mechanism_override` allow arbitrary mechanism replacement. All intervention types are immutable: they are applied to a copy of the system, leaving the original unchanged and available for comparison.

4 Mechanism library

4.1 Pre-built factories

The **lepidoptera** mechanism library provides one factory function for each of the nine parent-type/child-type combinations that arise from the three variable types. All factories are accessible through `mechanism_library_get()` using a string key, or directly by name. For a continuous child with continuous parents, `linear_gaussian()` implements $Y = \alpha + \boldsymbol{\beta}^\top \mathbf{X} + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, \sigma^2)$ (Equation 1). For a categorical child with continuous parents, `multinomial_logit()` implements the standard log-linear model (Equation 2) (Agresti, 2002). For an ordinal child with continuous parents, `ordered_logit()` implements the proportional-odds cumulative link model (McCullagh, 1980) (Equation 3). When the parent is categorical or ordinal and the child is continuous, `group_effect()` samples from a group-specific Gaussian $Y \mid X = k \sim \mathcal{N}(\mu_k, \sigma_k^2)$. For categorical or ordinal children with categorical or ordinal parents, `categorical_transition()` uses a full conditional probability table. Two cross-cutting mechanisms, `copula_mechanism()` and `model_mechanism()`, are described in Sections 6 and 7 respectively and can be used for any type combination.

$$Y = \alpha + \sum_{j=1}^{q} \beta_j X_j + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2) \tag{1}$$

$$P(Y = k \mid \mathbf{X}) = \frac{\exp(\alpha_k + \boldsymbol{\beta}_k^\top \mathbf{X})}{\sum_{\ell=1}^{K} \exp(\alpha_\ell + \boldsymbol{\beta}_\ell^\top \mathbf{X})} \tag{2}$$

$$\log \frac{P(Y \preceq \ell_k \mid \mathbf{X})}{P(Y \succ \ell_k \mid \mathbf{X})} = \gamma_k - \boldsymbol{\beta}^\top \mathbf{X}, \quad \gamma_1 < \cdots < \gamma_{K-1} \tag{3}$$

4.2 Building a system: a worked example

The following example constructs a four-variable causal system representing an epidemiological mediation structure: a continuous income variable acts as a root node; income determines smoking status (categorical) through a multinomial logit; income also determines diet score (continuous) through a linear Gaussian relationship; and both smoking status and diet score jointly determine cardiovascular disease risk (categorical) through a second multinomial logit. The design follows the mediation framework of VanderWeele (2015).

```
library("lepidoptera")

income_spec  <- feature_spec("income",     "continuous",
                             list(min = 0, max = Inf))
smoke_spec   <- feature_spec("smoking",    "categorical",
                             list(levels = c("never","former","current")))
diet_spec    <- feature_spec("diet_score", "continuous",
                             list(min = 0, max = 100))
cvd_spec     <- feature_spec("cvd_risk",   "categorical",
                             list(levels = c("low","high")))

mech_income <- mechanism_library_get("normal")(mean = 50, sd = 15)
mech_smoke  <- multinomial_logit(
  levels     = c("never","former","current"),
  coefs      = list(former  = list(income = -0.03),
                    current = list(income = -0.05)),
  intercepts = list(former  =  0.2, current = -0.8)
)
mech_diet   <- linear_gaussian(
  coefs = list(income = 0.6), intercept = 20, sd = 8
)
mech_cvd    <- multinomial_logit(
  levels     = c("low","high"),
  coefs      = list(high = list(diet_score      = -0.04,
                                smoking_former  =  0.50,
                                smoking_current =  1.20)),
  intercepts = list(high = -2.0)
)

mediation_sys <- new_system()                                          |>
  add_feature(income_spec) |> add_feature(smoke_spec)                  |>
  add_feature(diet_spec)   |> add_feature(cvd_spec)                    |>
  add_dependency(character(0), "income",    mech_income)               |>
  add_dependency("income",     "smoking",   mech_smoke)                |>
  add_dependency("income",     "diet_score",mech_diet)                 |>
  add_dependency(c("diet_score","smoking"), "cvd_risk", mech_cvd)      |>
  finalize_system()
```

```
System finalised: 4 features, 5 edges.
Topological order: income -> smoking -> diet_score -> cvd_risk
All 5 invariants passed.
```

The system is immediately ready for simulation. The call `simulate(mediation_sys, n = 2000, seed = 42)` returns a data frame of 2 000 rows in topological order, with types enforced throughout. Because the mechanism objects are stored separately from the graph, each one can be inspected (`mech_diet$spec`) or replaced via `mechanism_override( "diet_score", new_mech)` without touching the remaining nodes.

5 Simulation and propagation

5.1 Population simulation

Population simulation proceeds in two steps. First, `finalize_system()` computes a topological ordering of G using Kahn's algorithm (Kahn, 1962). Second, for unit $i = 1, \ldots, n$, the global RNG seed is set to $s + i$ and the nodes are sampled in topological order: each node V_k receives the already-computed realisations of its parents and calls its mechanism's sampling function. The per-unit seed $s + i$ is the key reproducibility device: it ensures that any single unit can be regenerated exactly by re-running with the same seed, and it enables the RNG-state freezing that partial propagation exploits.

5.2 Two modes of intervention propagation

lepidoptera provides two propagation modes, selected through the `mode` argument of `propagate()`. Under `mode = "full"`, the intervention is applied to a copy of the system and simulation is re-run from scratch with the same global seed s. Because per-unit seeds are identical to the baseline run, units are paired one-to-one by construction, and the sample ATE is $n^{-1} \sum_i (Y_i^{\text{int}} - Y_i^{\text{base}})$. This mode assumes nothing beyond SUTVA (Rubin, 1980) and yields unbiased estimates of $E[Y^{do(t)}]$ in the sense of Rubin (1974): it produces independent draws from the joint interventional distribution.

Under `mode = "partial"`, the package first identifies the *affected set* $\text{Aff}(\mathcal{T}, G)$—the transitive downstream closure of the intervention targets $\mathcal{T}$ in G—and then, for each unit i, sets the RNG seed to $s + i$ (identical to the baseline pass), freezes the drawn values for all nodes *not* in Aff, and resamples only the nodes in Aff under the modified mechanism. Because unaffected nodes retain their exact baseline draws, the counterfactual and baseline populations share the same exogenous noise realisations, operationalising Pearl's abduction-action-prediction procedure (Pearl, 2009) without ever explicitly representing the background-variable distribution. It is straightforward to verify that, under this procedure, (i) every node outside Aff takes the same value in the counterfactual population as in the baseline population, and (ii) every node inside Aff is sampled from its modified mechanism applied to its counterfactual parent values. The sample ITE for unit i is then simply $Y_i^{\text{cf}} - Y_i^{\text{base}}$.

These two modes answer qualitatively different questions. `mode = "full"` is appropriate for policy evaluation—what is the average effect of setting treatment T to t across the population?—and for power calculations that need independent draws from the interventional distribution. `mode = "partial"` is appropriate for individual explanation and attribution—for this specific patient, what would the outcome have been under a different treatment?—and for any analysis where reducing variance by pairing observations on their shared exogenous noise is valuable. The choice between modes does not depend on the variable types or mechanisms in the system; both modes work with any combination of pre-built and user-supplied mechanisms.

5.3 A counterfactual query

Building on the mediation system from Section 4, the following code asks: among patients who are currently at high cardiovascular risk, what would their risk status be if income were fixed at 80 for everyone?

```
baseline   <- simulate(mediation_sys, n = 1000, seed = 42)
cf_result  <- propagate(
  mediation_sys,
  interventions = list(do_fix("income", 80)),
  n = 1000, seed = 42, mode = "partial"
)
high_risk <- baseline$cvd_risk == "high"
cf_risk   <- cf_result$intervened$cvd_risk[high_risk]
cat(sprintf(
  "Of %d high-risk patients, under income = 80: %d (%.0f%%) would become
    low-risk\n",
  sum(high_risk), sum(cf_risk == "low"), mean(cf_risk == "low") * 100
))
```

```
Of 208 high-risk patients, under income = 80: 87 (42%) would become low-risk
```

The result is an individual-level statement: for the same 208 individuals who appeared at high risk in the baseline simulation, 42% would have been at low risk had their income been higher, holding all exogenous noise fixed. The complementary `mode = "full"` call with the same seed would have yielded a population-level average, using new independent draws rather than the same individuals.

6 Vine copula mechanism

6.1 Motivation

The mechanisms described in Section 4 encode structural equations: a parametric functional form maps parent values to a child distribution. When an analyst has observed data on a group of variables and wishes to preserve their joint dependence structure—including tail dependence, asymmetry, and multivariate co-movement that parametric forms would miss—specifying a structural equation by hand is both burdensome and potentially misleading. The vine copula mechanism addresses this by estimating the full dependence structure nonparametrically from data, via pair-copula constructions (Bedford and Cooke, 2002; Aas et al., 2009), and using the resulting model as the conditional generating rule at simulation time. The theoretical foundation is Sklar's theorem (Sklar, 1959), which decomposes any joint distribution into its marginals and a copula that captures all dependence. The approach inherits the rich family vocabulary of **rvinecopulib** (Nagler and Vatter, 2022), including the Gaussian and Student-t elliptical copulas, the Archimedean families—Clayton (Clayton, 1978), Gumbel (Gumbel, 1960), Frank (Frank, 1979), and Joe (Joe, 1993)—and their two-parameter generalisations, as well as a non-parametric kernel smoother (Nagler and Czado, 2016) (`"tll"`) and automatic family selection by AIC (Czado, 2019) (`"auto"`). Tail-dependence coefficients for the selected families are available through **rvinecopulib**'s underlying **vinecopulib** C++ library (Joe, 2014).

6.2 Algorithm

lepidoptera uses the D-vine structure (Dißmann et al., 2013), which orders variables linearly. The child V_i is always placed last in the ordering $(\sigma_1, \ldots, \sigma_{d-1}, i)$, so that the first tree of the vine contains pair copulas between consecutive variables in the chain and

higher trees condition on progressively more variables. Conditional simulation exploits the Rosenblatt transform (Rosenblatt, 1952): given a fitted D-vine $\hat{C}$, the parent pseudo-observations $(\hat{u}_1, \ldots, \hat{u}_{d-1})$ computed from the empirical CDF of the training data, and a fresh draw $w \sim \text{Uniform}(0, 1)$, the algorithm (i) evaluates the first $d-1$ components of the Rosenblatt transform at the observed parent values (the placeholder value used for the child position does not propagate into these components), (ii) replaces the d-th component with w, (iii) evaluates the inverse Rosenblatt transform to obtain the child uniform u_i^*, and (iv) maps u_i^* back to the original scale through the empirical quantile function. In the bivariate case ($d = 2$) this reduces to evaluating the inverse h-function (Aas et al., 2009), which **rvinecopulib** provides directly through `hbicop(..., inverse = TRUE)`. The bivariate path is used whenever a single parent is specified, avoiding the overhead of the full Rosenblatt pipeline.

6.3 Example

```
set.seed(1)
obs <- data.frame(age = rnorm(400, 62, 10),
                  comorbidity = rnorm(400, 4, 1.5))
obs$comorbidity <- obs$comorbidity -
  0.35 * (obs$age - 62) / 10 + rnorm(400, 0, 0.8)

mech_comorbidity <- copula_mechanism(
  variables = c("comorbidity", "age"),
  family    = "auto",
  fit_data  = obs
)

mech_age <- mechanism_library_get("normal")(mean = 62, sd = 10)
sys_cop  <- new_system()                                          |>
  add_feature(feature_spec("age", "continuous",
                           list(min = 18, max = 120)))            |>
  add_feature(feature_spec("comorbidity", "continuous",
                           list(min = 0, max = Inf)))             |>
  add_dependency(character(0), "age",         mech_age)          |>
  add_dependency("age",        "comorbidity", mech_comorbidity)  |>
  finalize_system()

sim_cop <- simulate(sys_cop, n = 800, seed = 99)
cat(sprintf("Observed rho_S = %.2f   Simulated rho_S = %.2f\n",
  cor(obs$age, obs$comorbidity,         method = "spearman"),
  cor(sim_cop$age, sim_cop$comorbidity, method = "spearman")))
```

```
Observed rho_S = -0.31   Simulated rho_S = -0.29
```

The simulated Spearman correlation is close to the observed value, confirming that the vine copula mechanism has captured the negative dependence between age and comorbidity burden. When `family = "auto"` is selected, **rvinecopulib** chooses the best-fitting bivariate copula family for each pair in the vine by AIC, so the mechanism adapts automatically to the data's tail-dependence structure without requiring the user to specify a family in advance.

Vine copulas require a total order on marginals. For nominal categorical variables, which have no natural order, **lepidoptera** exposes this choice through the `nominal_handling` argument of `copula_mechanism()`: the default `"regression"` raises an error if a nominal variable appears in `variables`, directing the user to a parametric mechanism; the `"arbitrary_order"` option includes it with a user-specified level mapping and emits a warning at `finalize_system()` time. This design reflects the view that the package should not make silent arbitrary choices on the user's behalf (Pearl, 2009).

7 Predictive model integration

The mechanism contract—any callable with signature `function(parent_values) -> value`—is already the natural integration boundary for machine-learning models. This reflects a broader trend in causal inference towards incorporating flexible, data-driven components within a causal graph structure (Schölkopf et al., 2021; Peters et al., 2017). Three wrapper functions make common patterns idiomatic and add type validation without constraining the user's prediction logic.

`model_mechanism()` wraps any pre-fitted R or Python model. The user supplies a `predict_fn` closure that accepts a named list of parent values and returns a single draw from the child distribution; the package validates types and propagates the mechanism through the standard simulation and intervention machinery. The following example wraps a **ranger** random forest (Wright and Ziegler, 2017; Breiman, 2001) to serve as a probabilistic mechanism for a binary treatment response:

```
rf <- ranger::ranger(response ~ age + dose + biomarker,
                     data = trial_data, probability = TRUE,
                     num.trees = 500)

mech_response <- model_mechanism(
  predict_fn   = function(pv) {
    nd   <- data.frame(age       = as.numeric(pv$age),
                       dose      = as.numeric(pv$dose),
                       biomarker = as.numeric(pv$biomarker))
    prob <- predict(rf, nd)$predictions[1, ]
    sample(c("no_response","response"), 1, prob = prob)
  },
  parents      = c("age","dose","biomarker"),
  parent_types = c("continuous","continuous","continuous"),
  output_type  = "categorical",
  levels       = c("no_response","response")
)
```

The user controls all prediction logic inside `predict_fn`: input reshaping, output slot selection, and optional residual noise injection. From **lepidoptera**'s perspective this mechanism is identical to any pre-built factory: it participates in topological ordering, type validation, intervention propagation, and counterfactual resampling. Other supported backends include **xgboost**, **glm**, **lm**, **nnet**, **keras**, and **torch**.

`learned_mechanism()` takes a higher-level approach: it accepts a data frame with both parent and child columns and fits a mechanism automatically, using OLS with Gaussian residuals for continuous children and a k-nearest-neighbour empirical sampler for categorical and ordinal children. This is convenient for rapid prototyping when observational data are

available but the analyst does not wish to specify a functional form. `llm_mechanism()` routes the prediction step through a user-supplied language-model client, accepting a prompt template with `{variable}` placeholders; because LLM calls are expensive and non-deterministic, `finalize_system()` emits a warning when an `llm_mechanism()` is on a high-frequency path and suggests memoisation for parents with a finite discrete domain.

8 Illustrations

8.1 Clinical trial simulation with confounded treatment assignment

A common application of causal simulation in biostatistics is the generation of synthetic patient populations for power calculations and for demonstrating the performance of estimators under confounding. The following illustration constructs a Phase II oncology trial scenario (full system specification is available in the replication script `lepidoptera-paper.R` included with this paper) in which patient age and ECOG performance status jointly confound the treatment-response relationship. Ordinal ECOG scores are generated from age via `ordered_logit()`; treatment assignment follows a multinomial logit with age and ECOG as predictors; response is generated from treatment, ECOG, and age. Under the observational (confounded) regime, patients with better performance status are more likely to receive active treatment and also more likely to respond, so the naive odds ratio overstates the treatment effect.

```
obs_pop <- simulate(trial_sys, n = 5000, seed = 1)
rct_pop <- propagate(
  trial_sys,
  interventions = list(do_distribution("treatment",
    categorical_marginal(c("control","active"), c(0.5, 0.5)))),
  n = 5000, seed = 1, mode = "full"
)
naive_or <- with(obs_pop,
  mean(response[treatment == "active"]  == "yes") /
  mean(response[treatment == "control"] == "yes"))
rct_or   <- with(rct_pop$intervened,
  mean(response[treatment == "active"]  == "yes") /
  mean(response[treatment == "control"] == "yes"))
cat(sprintf("Naive OR = %.2f   Adjusted OR = %.2f\n", naive_or, rct_or))
```

```
Naive OR = 1.84   Adjusted OR = 1.41
```

The naive observational odds ratio overstates the treatment effect by about 30% relative to the do-operator estimate, consistent with the direction of confounding embedded in the mechanism specification. The separation of population-level (`mode = "full"`) propagation from individual-level (`mode = "partial"`) propagation means the analyst can compute both the ATE and the individual counterfactuals from the same system object without any re-specification.

8.2 Emergency-ward digital twin with a triage intervention

A hospital data scientist models an emergency ward as an SCM with five nodes (full system specification in `lepidoptera-paper.R`): continuous inter-arrival time, ordinal triage priority (P1/P2/P3 via `ordered_logit()` on arrival acuity), continuous length of stay fitted from historical discharge data via `copula_mechanism()`, and categorical discharge destination (home/admitted/transferred via `multinomial_logit()`). The individual-level research question is: for patients who were discharged home, what would their discharge destination have been had triage priority been upgraded by one level? This is a counterfactual on the ordinal triage node, propagated through the vine copula length-of-stay mechanism to the categorical discharge node.

```
upgrade_fn <- function(x) c(P3="P2", P2="P1", P1="P1")[x]
ward_cf    <- propagate(
  ward_sys,
  interventions = list(shift("triage", upgrade_fn)),
  n = 2000, seed = 7, mode = "partial"
)
home_pts <- ward_cf$baseline$discharge == "home"
print(prop.table(table(ward_cf$intervened$discharge[home_pts])) * 100)
```

```
   admitted      home transferred
      18.4%     79.1%       2.5%
```

Approximately 18% of patients who were in fact discharged home would have been admitted had they received a higher triage priority, providing actionable quantification for capacity-planning scenarios. Because `mode = "partial"` was used, the result refers to the same individuals who appeared as home-discharged in the baseline, not a newly drawn population.

8.3 Mediation analysis: natural direct and indirect effects

The mediation system from Section 4 is used to decompose the total effect of income on cardiovascular disease risk into a natural direct effect (NDE) and a natural indirect effect (NIE) mediated through smoking and diet (Imai et al., 2010; Imai and Yamamoto, 2013), using the cross-world intervention representation of Robins and Greenland (1992) and Pearl (2001). The NDE is estimated by fixing income at the high level while also fixing the mediators at their values under the low-income regime.

```
n <- 5000; seed <- 42
r_low  <- propagate(mediation_sys, list(do_fix("income", 30)),
                    n = n, seed = seed, mode = "full")
r_high <- propagate(mediation_sys, list(do_fix("income", 80)),
                    n = n, seed = seed, mode = "full")
med_low <- r_low$intervened[, c("smoking","diet_score")]
r_nde   <- propagate(mediation_sys,
             list(do_fix("income",     80),
                  do_fix("smoking",    med_low$smoking),
                  do_fix("diet_score", med_low$diet_score)),
             n = n, seed = seed, mode = "full")
te  <- mean(r_high$intervened$cvd_risk == "high") -
       mean(r_low$intervened$cvd_risk  == "high")
```

```
nde <- mean(r_nde$intervened$cvd_risk  == "high") -
       mean(r_low$intervened$cvd_risk  == "high")
nie <- te - nde
cat(sprintf("TE = %.3f  NDE = %.3f  NIE = %.3f  Prop. mediated = %.1f%%\n",
            te, nde, nie, nie/te*100))
```

```
TE = -0.147  NDE = -0.089  NIE = -0.058  Prop. mediated = 39.5%
```

Moving from low to high income reduces the probability of high cardiovascular risk by about 15 percentage points in total; 39.5% of this reduction operates through the smoking and diet mediators. These values are consistent with the direction and approximate magnitude discussed in VanderWeele (2015), and the code demonstrates that mediation decompositions requiring cross-world interventions are handled cleanly within **lepidoptera**'s intervention framework without additional specification.

9 Computational details

lepidoptera is available from GitHub at `https://github.com/daniloasarti/lepidoptera` under the MIT licence (GPL-2 compatible). The package is written in R 4.4.0 (R Core Team, 2024) and optionally links to **rvinecopulib** 0.6.3 (Nagler and Vatter, 2022) for the vine copula mechanism; all other dependencies are listed in the package `DESCRIPTION` file. A Python implementation using **pyvinecopulib** 0.6 (Nagler and Vatter, 2024) is available in the same repository under the `python/` subdirectory. Population simulation runs in $\mathcal{O}(np)$ time where n is the population size and p is the number of features; the vine copula mechanism adds an $\mathcal{O}(nd^2)$ term per node from the Rosenblatt transform, where d is the number of parents. All results in this paper were obtained on macOS 14 (Apple M-series, 16 GB RAM). The replication script `lepidoptera-paper.R`, which reproduces all code outputs in this paper, is included with the submission.

10 Summary and discussion

lepidoptera provides a unified interface for causal simulation with heterogeneous variable types, data-driven dependence modelling, and individual-level counterfactual propagation. The typed mechanism architecture ensures that type errors are caught at system-finalisation time rather than during simulation; the pre-built library removes the need to implement type-specific sampling from scratch; the vine copula mechanism brings nonparametric dependence modelling into the SCM workflow; and partial propagation enables ITE computation from the same synthetic population as the ATE. The package complements rather than replaces existing tools: **simDAG** (Denz and Timmesfeld, 2026) excels at longitudinal discrete-time simulation with complex event-time outcomes; **dagitty** (Textor et al., 2016) handles structural identification; **cfid** (Tikka et al., 2021) addresses counterfactual query identification.

Several limitations of the current version merit acknowledgement. The current version restricts G to a DAG; feedback loops and temporal dynamics are planned for a future release, following the unrolled-graph architecture of Bongers et al. (2021) and the longitudinal framework of **simcausal**. The vine copula mechanism requires complete training data; a multiple-imputation wrapper for missing data is under development. LLM mechanism calls are expensive and non-deterministic even with a fixed seed; the built-in memoisation cache

mitigates cost for finite-domain parents but is not a complete solution. Finally, the `mode = "partial"` counterfactual requires that the noise model be at least implicitly additive, in the sense that freezing the RNG stream across the baseline and counterfactual populations corresponds to holding the exogenous noise fixed; for mechanisms with substantially non-additive noise structures, the more general algorithm of Karvanen et al. (2024) may be preferable. Future directions include tighter integration with **dagitty** for identification-aware simulation, GPU-accelerated batch simulation, an interactive DAG editor, and differential privacy wrappers for disclosure-limited synthetic data release (Nowok et al., 2016; Reiter, 2002).

Acknowledgements

The author thanks the developers of **rvinecopulib** (Nagler and Vatter, 2022) and **simcausal** (Sofrygin et al., 2017), whose architectures informed several design decisions.

References

Kjersti Aas, Claudia Czado, Arnoldo Frigessi, and Henrik Bakken. Pair-copula constructions of multiple dependence. *Insurance: Mathematics and Economics*, 44(2):182–198, 2009. doi: 10.1016/j.insmatheco.2007.02.001.

Alan Agresti. *Categorical Data Analysis*. Wiley, Hoboken, NJ, 2nd edition, 2002.

Ghadi S. Al Hajj, Muhammad Umer Gul, Lasse Labrijn Bongo, Jonas Christoffer Lindstrøm, and Lisa Harber-Aschan. **DagSim**: Concept and Python package for DAG-based data simulation with mixed variable types and complex functional relationships. *PLOS ONE*, 18(5):e0284443, 2023. doi: 10.1371/journal.pone.0284443.

Paul Beaumont, Ben Horsburgh, Philip Pilgerstorfer, Angel Droth, Richard Oentaryo, Steven Ler, Hiep Nguyen, Gabriel Ferber, Alexis Lerousseau, and Andrew Sharp. **CausalNex**: A Python library for causal reasoning and Bayesian networks. `https://github.com/quantumblacklabs/causalnex`, 2021.

Tim Bedford and Roger M. Cooke. Vines: A new graphical model for dependent random variables. *Annals of Statistics*, 30(4):1031–1068, 2002. doi: 10.1214/aos/1031689016.

Stephan Bongers, Patrick Forré, Jonas Peters, and Joris M. Mooij. Foundations of structural causal models with cycles and latent variables. *Annals of Statistics*, 49(5): 2885–2915, 2021.

Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.

Andrea Burton, Douglas G. Altman, Patrick Royston, and Roger L. Holder. The design of simulation studies in medical statistics. *Statistics in Medicine*, 25(24):4279–4292, 2006. doi: 10.1002/sim.2673.

David Maxwell Chickering. Optimal structure identification with greedy search. *Journal of Machine Learning Research*, 3:507–554, 2002.

David G. Clayton. A model for association in bivariate life tables and its application in epidemiological studies of familial tendency in chronic disease incidence. *Biometrika*, 65 (1):141–151, 1978.

Claudia Czado. *Analyzing Dependent Data with Vine Copulas*. Springer, Cham, 2019. doi: 10.1007/978-3-030-13785-4.

Robin Denz and Nina Timmesfeld. Simulating data from directed acyclic graphs using the **simDAG** package for R. *Journal of Statistical Software*, 116(2):1–38, 2026. doi: 10.18637/jss.v116.i02.

Jeffrey Dißmann, Eike Christian Brechmann, Claudia Czado, and Dorota Kurowicka. Selecting and estimating regular vine copulae and application to financial returns. *Computational Statistics & Data Analysis*, 59:52–69, 2013. doi: 10.1016/j.csda.2012.08.010.

Maurice J. Frank. On the simultaneous associativity of $f(x, y)$ and $x + y - f(x, y)$. *Aequationes Mathematicae*, 19(1):194–226, 1979.

Keith S. Goldfeld and Jacob Wujciak-Jens. **simstudy**: Illuminating research methods through data generation. *Journal of Open Source Software*, 5(54):2763, 2020. doi: 10.21105/joss.02763.

Emil J. Gumbel. Bivariate exponential distributions. *Journal of the American Statistical Association*, 55(292):698–707, 1960.

Kosuke Imai and Teppei Yamamoto. Identification and sensitivity analysis for multiple causal mechanisms: Revisiting evidence from framing experiments. *Political Analysis*, 21(2):141–171, 2013. doi: 10.1093/pan/mps040.

Kosuke Imai, Luke Keele, and Teppei Yamamoto. Identification, inference and sensitivity analysis for causal mediation effects. *Statistical Science*, 25(1):51–71, 2010. doi: 10.1214/10-STS321.

Harry Joe. Parametric families of multivariate distributions with given margins. *Journal of Multivariate Analysis*, 46(2):262–282, 1993. doi: 10.1006/jmva.1993.1061.

Harry Joe. *Dependence Modeling with Copulas*. Chapman & Hall/CRC, Boca Raton, FL, 2014.

Arthur B. Kahn. Topological sorting of large networks. *Communications of the ACM*, 5 (11):558–562, 1962. doi: 10.1145/368996.369025.

Juha Karvanen, Santtu Tikka, and Matti Vihola. Simulating counterfactuals. *Journal of Artificial Intelligence Research*, 80:1–38, 2024. doi: 10.1613/jair.1.15381.

Reinhard Laubenbacher, James P. Sluka, and James A. Glazier. Digital twins in medicine. *Nature Computational Science*, 4:164–171, 2024. doi: 10.1038/s43588-024-00609-5.

Steffen L. Lauritzen and Nanny Wermuth. Graphical models for associations between variables, some of which are qualitative and some quantitative. *Annals of Statistics*, 17 (1):31–57, 1989. doi: 10.1214/aos/1176347003.

Peter McCullagh. Regression models for ordinal data. *Journal of the Royal Statistical Society, Series B*, 42(2):109–142, 1980.

Tim P. Morris, Ian R. White, and Michael J. Crowther. Using simulation studies to evaluate statistical methods. *Statistics in Medicine*, 38(11):2074–2102, 2019. doi: 10.1002/sim.8086.

Thomas Nagler and Claudia Czado. Evading the curse of dimensionality in nonparametric density estimation with simplified vine copulas. *Journal of Multivariate Analysis*, 151: 69–89, 2016. doi: 10.1016/j.jmva.2016.07.003.

Thomas Nagler and Thibault Vatter. **rvinecopulib**: High performance algorithms for vine copula modeling. *Journal of Statistical Software*, 102(5):1–34, 2022. doi: 10.18637/jss.v102.i05.

Thomas Nagler and Thibault Vatter. **pyvinecopulib**: Python bindings for **vinecopulib**. https://github.com/vinecopulib/pyvinecopulib, 2024. Python package version 0.6.

Michael C. Neale, Michael D. Hunter, Joshua N. Pritikin, Mahsa Zahery, Timothy R. Brick, Robert M. Kirkpatrick, Ryne Estabrook, Timothy C. Bates, Hermine H. Maes, and Steven M. Boker. **OpenMx** 2.0: Extended structural equation and statistical modeling. *Psychometrika*, 81(2):535–549, 2016. doi: 10.1007/s11336-014-9435-8.

Jerzy Neyman. On the application of probability theory to agricultural experiments. Essay on principles. Section 9. *Statistical Science*, 5(4):465–472, 1990. Translated from the Polish original (1923).

Beata Nowok, Gillian M. Raab, and Chris Dibben. **synthpop**: Bespoke creation of synthetic data in R. *Journal of Statistical Software*, 74(11):1–26, 2016. doi: 10.18637/jss.v074.i11.

Judea Pearl. Causal diagrams for empirical research. *Biometrika*, 82(4):669–688, 1995.

Judea Pearl. Direct and indirect effects. *Proceedings of the Seventeenth Conference on Uncertainty in Artificial Intelligence*, pages 411–420, 2001.

Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, Cambridge, 2nd edition, 2009.

Jonas Peters, Dominik Janzing, and Bernhard Schölkopf. *Elements of Causal Inference: Foundations and Learning Algorithms*. MIT Press, Cambridge, MA, 2017.

R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2024. URL https://www.R-project.org/.

Jerome P. Reiter. Satisfying disclosure restrictions with synthetic data sets. *Journal of Official Statistics*, 18(4):531–544, 2002.

James M. Robins and Sander Greenland. Identifiability and exchangeability for direct and indirect effects. *Epidemiology*, 3(2):143–155, 1992.

James M. Robins, Miguel A. Hernan, and Babette Brumback. Marginal structural models and causal inference in epidemiology. *Epidemiology*, 11(5):550–560, 2000.

Murray Rosenblatt. Remarks on a multivariate transformation. *Annals of Mathematical Statistics*, 23(3):470–472, 1952. doi: 10.1214/aoms/1177729394.

Yves Rosseel. **lavaan**: An R package for structural equation modeling. *Journal of Statistical Software*, 48(2):1–36, 2012. doi: 10.18637/jss.v048.i02.

Donald B. Rubin. Estimating causal effects of treatments in randomized and nonrandomized studies. *Journal of Educational Psychology*, 66(5):688–701, 1974.

Donald B. Rubin. Randomization analysis of experimental data: The Fisher randomization test comment. *Journal of the American Statistical Association*, 75(371):591–593, 1980.

Bernhard Schölkopf, Francesco Locatello, Stefan Bauer, Nan Rosemary Ke, Nal Kalchbrenner, Anirudh Goyal, and Yoshua Bengio. Toward causal representation learning. *Proceedings of the IEEE*, 109(5):612–634, 2021. doi: 10.1109/JPROC.2021.3058954.

Marco Scutari. Learning Bayesian networks with the **bnlearn** R package. *Journal of Statistical Software*, 35(3):1–22, 2010. doi: 10.18637/jss.v035.i03.

Amit Sharma and Emre Kiciman. **DoWhy**: An end-to-end library for causal inference. https://arxiv.org/abs/2011.04216, 2020.

Abe Sklar. Fonctions de répartition à n dimensions et leurs marges. *Publications de l'Institut de Statistique de l'Université de Paris*, 8:229–231, 1959.

Oleg Sofrygin, Mark J. van der Laan, and Romain Neugebauer. **simcausal** R package: Conducting transparent and reproducible simulation studies of causal effect estimation with complex longitudinal data. *Journal of Statistical Software*, 81(2):1–47, 2017. doi: 10.18637/jss.v081.i02.

Peter Spirtes, Clark Glymour, and Richard Scheines. *Causation, Prediction, and Search*. MIT Press, Cambridge, MA, 2nd edition, 2000.

Johannes Textor, Benito van der Zander, Mark S. Gilthorpe, Maciej Liśkiewicz, and George T.H. Ellison. Robust causal inference using directed acyclic graphs: the R package **dagitty**. *International Journal of Epidemiology*, 45(6):1887–1894, 2016. doi: 10.1093/ije/dyw341.

Santtu Tikka. **R6causal**: R6 classes for structural causal models. https://CRAN.R-project.org/package=R6causal, 2024. R package version 0.8.3.

Santtu Tikka, Antti Hyttinen, and Juha Karvanen. Causal effect identification from multiple incomplete data sources: A general search-based approach. *Journal of Statistical Software*, 99(5):1–40, 2021. doi: 10.18637/jss.v099.i05.

Mark J. van der Laan and Sherri Rose. *Targeted Learning in Data Science: Causal Inference for Complex Longitudinal Studies*. Springer, New York, 2018.

Tyler J. VanderWeele. *Explanation in Causal Inference: Methods for Mediation and Interaction*. Oxford University Press, New York, 2015.

Thomas S. Verma and Judea Pearl. Causal networks: Semantics and expressiveness. Technical report, UCLA Cognitive Systems Laboratory, 1988. Technical Report R-65.

Marvin N. Wright and Andreas Ziegler. **ranger**: A fast implementation of random forests for high dimensional data in C++ and R. *Journal of Statistical Software*, 77(1):1–17, 2017. doi: 10.18637/jss.v077.i01.